Modelagem e Ingestão de Dados

Seminário BDs NoSQL – Grupo 5 – Banco multimodelo ArangoDB – Parte 4 (~4 min, 3 slides)

1. Ideia central

Em bancos NoSQL, **modelar não é normalizar tabelas: é decidir como os dados serão guardados a partir das consultas que a aplicação precisa responder**. A pergunta de partida deixa de ser "quais entidades e chaves estrangeiras existem?" e passa a ser "o que eu preciso buscar, e com que frequência?".

No nosso projeto o cenário é uma **rede social**. Ela tem três necessidades distintas: perfis com dados flexíveis, relações de amizade e sessões temporárias de login. Em vez de criar três bancos diferentes (persistência poliglota), usamos o ArangoDB, que oferece os três modelos no mesmo motor e na mesma linguagem de consulta (AQL).

2. Modelagem relacional x NoSQL

Para a mesma rede social, as abordagens seriam estas:

Aspecto	Relacional (ex.: PostgreSQL)	ArangoDB (multimodelo)
Perfil do usuário	Tabela com colunas fixas; dado novo exige ALTER TABLE ou colunas nulas; lista de hobbies pede outra tabela	Documento JSON; cada usuário pode ter campos diferentes e listas embutidas
Amizade	Tabela N:N com duas chaves estrangeiras	Aresta (edge) em uma coleção de arestas, com propriedades próprias
Amigos de amigos	JOINs encadeados, cada nível acrescenta um JOIN	Travessia de grafo nativa (uma linha de AQL, profundidade configurável)
Sessão	Tabela comum, sem otimização específica	Coleção usada como chave-valor, acesso direto pela chave
Esquema	Definido antes de inserir (schema-on-write)	Flexível (schemaless); a estrutura vem dos próprios dados

3. O modelo adotado: três coleções, três modelos

Cada parte do problema foi para o modelo que melhor combina com ela:

Coleção	Modelo	Tipo no ArangoDB	Por que esse modelo
usuarios	Documento	Document collection	Perfil tem dados flexíveis (hobbies é lista, campos podem variar entre pessoas)
amizades	Grafo	Edge collection	O relacionamento é o dado principal; consultas de vizinhança e caminho são nativas
sessoes	Chave-valor	Document collection usada por chave	Busca direta pela `_key`, dado temporário e simples

O grafo `rede_social` é a definição que liga `usuarios` (vértices) e `amizades` (arestas) e permite visualizar a rede no painel web.

4. Anatomia dos dados

4.1 Campos automáticos

Todo documento no ArangoDB recebe três atributos de sistema:

- `_key`: identificador do documento dentro da coleção. Pode ser definido manualmente (ex.: `u01`) ou gerado pelo banco.
- `_id`: identificador único no banco inteiro, formado por `colecao/_key` (ex.: `usuarios/u01`).
- `_rev`: versão do documento, atualizada a cada modificação; serve para controle de concorrência.

4.2 Documento (vértice): usuário

```
{
  "_key": "u01",
  "nome": "Alice",
  "idade": 28,
  "hobbies": ["leitura", "tecnologia"]
}
```

4.3 Aresta: amizade

Uma aresta é um documento especial que **obrigatoriamente** tem `_from` e `_to`, apontando para o `_id` de outros documentos. Qualquer outro campo é propriedade do relacionamento.

```
{
  "_from": "usuarios/u01",
  "_to":   "usuarios/u02",
  "desde": 2021
}
```

O campo `desde` pertence à *amizade*, não a nenhum dos dois usuários. No relacional isso exigiria colunas extras na tabela N:N; aqui é só mais um atributo da aresta.

4.4 Sessão (chave-valor)

```
{
  "_key": "sess_9f3a",
  "usuario": "usuarios/u01",
  "expira_em": "2026-11-10T18:00:00Z"
}
```

A `_key` funciona como a chave do par chave-valor: a consulta é uma busca direta por ela, sem varrer a coleção. *Observação:* os exemplos acima são ilustrativos; ajuste os nomes de campos aos arquivos reais do grupo.

5. Decisões de modelagem

5.1 Embutir ou referenciar?

Em bancos de documentos há duas formas de representar relações: **embutir** (guardar os dados relacionados dentro do próprio documento) ou **referenciar** (guardar o identificador do outro documento). A regra prática:

- **Embutir** quando o dado pertence ao documento e é lido junto com ele (ex.: a lista de `hobbies` dentro do usuário).
- **Referenciar** quando a relação é muitos-para-muitos, cresce sem limite ou precisa ser consultada de forma independente (ex.: amizades).

5.2 Por que amizades não ficam dentro do usuário?

Uma lista de IDs de amigos embutida no perfil funcionaria para consultas simples, mas ficaria ruim para **travessias de vários níveis**, para **caminho mais curto** e para guardar **propriedades da relação**. Como arestas, o banco mantém índices próprios para esse acesso e a consulta fica direta. Esse é o ponto onde o modelo de grafo se paga.

5.3 Desnormalização é aceitável

No NoSQL, repetir um pouco de informação para acelerar a leitura é uma escolha consciente, e não um erro de projeto. O custo é que a atualização passa a ser responsabilidade da aplicação. No nosso caso, a modelagem evita quase toda repetição: o perfil fica só em `usuarios` e as arestas apenas apontam para ele.

6. Ingestão dos dados

Ingestão é o processo de levar os dados de uma fonte (arquivos JSON ou CSV) para dentro das coleções. Como o ArangoDB aceita ambos os formatos, não é preciso escrever código de aplicação para importar.

6.1 Passo a passo

1. Criar as coleções com o tipo correto: `usuarios` e `sessoes` como coleções de documentos e `amizades` como **coleção de arestas (edge collection)**. O tipo é escolhido na criação e não muda depois; sem ele, o banco não trata os documentos como arestas.

2. Importar os dados, por um dos dois caminhos:

- **Interface web** (`localhost:8529`): aba Collections, abrir a coleção, opção Import, enviar o arquivo .json ou .csv. É o caminho mais visual, bom para a apresentação.
- **Linha de comando** com `arangoimport`: bom para automatizar e repetir o processo sempre que precisar voltar ao estado inicial.

3. Respeitar a ordem: importar `usuarios` antes de `amizades`, porque as arestas apontam para documentos que precisam existir.

6.2 Exemplo com arangoimport

```
# usuarios (documentos)
arangoimport --file usuarios.json --type json \
  --collection usuarios

# amizades (arestas): o tipo edge e informado na criacao
arangoimport --file amizades.json --type json \
  --collection amizades --create-collection true \
  --create-collection-type edge
```

Em `--type json` o arquivo é um array de objetos; para um objeto por linha, usa-se `jsonl`. Para CSV, `--type csv`. Se o servidor exigir autenticação, acrescentam-se as opções de usuário e banco (`--server.username`, `--server.database`).

6.3 Por que a ordem importa

O ArangoDB não obriga que `_from` e `_to` apontem para documentos existentes no momento da importação. Se as arestas entrarem antes dos usuários, ficam **arestas soltas** (dangling edges) e as consultas de grafo podem retornar resultados incompletos. Por isso a ordem é uma boa prática de ingestão, e não um detalhe.

6.4 Validar depois de importar

Uma checagem rápida em AQL confirma que a carga bateu com o esperado:

```
RETURN {
  usuarios: LENGTH(usuarios),
  amizades: LENGTH(amizades),
  sessoes:  LENGTH(sessoes)
}
```

Estado inicial da demonstração: **20 usuários, 30 amizades e nenhuma sessão**. Se a demo já foi executada antes, é preciso resetar a base. O usuário inserido durante a demonstração (Zeca) não pode existir antes, senão o primeiro INSERT dá erro de chave duplicada.

7. Perguntas que o professor pode fazer

Por que não colocar as amizades dentro do documento do usuário?

Funciona para casos simples, mas travessias de vários níveis, caminho mais curto e propriedades da relação ficam difíceis. Como arestas, o banco otimiza esse tipo de consulta.

O que acontece se as arestas forem importadas antes dos usuários?

Elas apontam para documentos inexistentes (arestas soltas) e as consultas de grafo podem retornar resultados incompletos.

Qual a diferença entre adaptar um CSV para relacional e para NoSQL?

No relacional, normaliza-se em tabelas e chaves. No NoSQL, decide-se o modelo a partir das consultas que se quer responder.

Por que usar três modelos em vez de só documentos?

Cada dado tem um padrão de acesso diferente: perfil flexível (documento), relacionamento (grafo) e acesso rápido por chave (chave-valor). Um único motor evita manter e sincronizar três bancos.

O que é uma edge collection?

Uma coleção especial cujos documentos têm `_from` e `_to`, representando ligações dirigidas entre documentos.

8. Glossário rápido

Termo	Significado
Schemaless	Sem esquema fixo: documentos da mesma coleção podem ter campos diferentes
Documento	Objeto JSON armazenado em uma coleção
Edge / aresta	Documento com `_from` e `_to` que liga dois outros documentos

Termo	Significado
Edge collection	Coleção que guarda arestas
Travessia	Percorrer o grafo a partir de um vértice, seguindo arestas por N níveis
Ingestão	Carga de dados externos (JSON/CSV) para dentro do banco
Persistência poliglota	Usar vários bancos, cada um com um modelo diferente, na mesma aplicação
Multimodelo	Um único motor de banco que suporta vários modelos de dados